

IOC ENRICHER

Application web Python d'enrichissement d'indicateurs de compromission (IOC) : API externes, score de risque justifié, base de données PostgreSQL et déploiement public.

Rapport de projet

Élément	Valeur
Auteur	Yannick Konan
Date	16 septembre 2026
Application en ligne	https://ioc-enricher-delta.vercel.app
Dépôt de code	https://github.com/tad-code/ioc-enricher
Langage principal	Python 3.11
Bibliothèques	Flask 3.1, requests, python-dotenv
Base de données	PostgreSQL hébergé sur Supabase
Hébergement	Vercel (fonction Python serverless)
API externes	ip-api.com, RDAP (rdap.org), CIRCL hashlookup

Sommaire

Rapport de projet.....	1
Sommaire.....	2
1. Présentation du projet.....	4
1.1 Contexte	4
1.2 Objectif	4
1.3 Périmètre.....	4
2. Analyse du besoin.....	4
3. Architecture de l'application	5
4. Choix techniques et justification	6
5. Les API externes.....	6
5.1 Sources retenues	6
5.2 Anatomie d'un appel.....	7
5.3 Exemple de réponse JSON et exploitation	7
5.4 Codes de statut HTTP gérés.....	7
6. Base de données.....	8
6.1 Pourquoi une base de données ?	8
6.2 Table ioc_analyses	8
6.3 Comment les données sont envoyées	8
6.4 Sécurité de l'accès.....	8
7. Interface web.....	8
8. Déploiement	13
8.1 Publication du code.....	13
8.2 Hébergement	13
8.3 Variables d'environnement	13
9. Sécurité : audit et corrections	13
10. Tests et validation	14
10.1 Tests automatisés.....	14
10.2 Résultats obtenus sur des indicateurs réels.....	14
10.3 Les six cas exigés	15
11. Difficultés rencontrées et solutions	15
12. Améliorations envisagées.....	15
13. Conclusion.....	15
Annexe A — Arborescence du projet.....	17
Annexe B — Extraits de code commentés	17
Annexe C — Commandes utiles.....	18

1. Présentation du projet

1.1 Contexte

Une équipe de cybersécurité reçoit en permanence des indicateurs techniques — adresses IP, noms de domaine, empreintes de fichiers — qu'il faut qualifier avant de décider d'une action : bloquer, surveiller ou ignorer.

Cette qualification, appelée enrichissement, oblige aujourd'hui l'analyste à ouvrir plusieurs sites spécialisés, à comparer des résultats hétérogènes et à recopier les informations dans un tableur. L'opération prend plusieurs minutes par indicateur et les recherches déjà effectuées ne sont conservées nulle part.

1.2 Objectif

L'objectif du projet est de fournir un outil unique qui, à partir d'une seule saisie, réponde automatiquement aux trois questions d'un analyste :

1. Quelle est la nature de cet indicateur et à qui appartient-il ?
2. Est-il dangereux, avec quel niveau de risque et pour quelles raisons ?
3. A-t-il déjà été analysé, et qu'avait-on conclu ?

L'application devait également être accessible publiquement sur Internet, sans installation locale, afin d'être démontrée en direct.

1.3 Périmètre

Le projet traite trois familles d'indicateurs : les adresses IP (IPv4 et IPv6), les noms de domaine et les empreintes de fichiers (MD5, SHA-1, SHA-256). Les indicateurs neutralisés, tels qu'on les trouve dans les rapports d'incident — 185[.]10[.]10[.]5 ou hxxps://evil[.]com — sont automatiquement remis en forme avant analyse.

Hors périmètre : l'analyse du contenu d'un fichier, l'analyse d'URL complètes en tant que telles, et la détection en temps réel sur un réseau.

2. Analyse du besoin

Le cahier des charges a été traduit en exigences techniques vérifiables :

Exigence	Traduction technique	Réalisation
Application développée en Python	Code organisé en modules, fonctions, dictionnaires, boucles, gestion des erreurs	app.py, ioc_parser.py, enrichment.py, scoring.py, db.py
Interface Web	Formulaire HTML + affichage serveur	Flask + gabarit Jinja2 (templates/index.html)
Au moins une API externe réellement utilisée	Appel HTTP + exploitation de la réponse	ip-api.com, RDAP, CIRCL hashlookup (+ AbuseIPDB en option)
Données récupérées au format JSON	Lecture et parcours d'objets JSON	reponse.json() puis extraction des champs
Base de données Supabase	Table créée, insertion, lecture, suppression	Table ioc_analyses, accès via l'API REST PostgREST
Dépôt GitHub	Code publié, secrets exclus	github.com/tad-code/ioc-enricher (.gitignore,

		.env.example)
Application déployée avec URL publique	Hébergement accessible sans installation	Vercel — https://ioc-enricher-delta.vercel.app
README complet	Documentation de 10 sections	README.md
Gestion des erreurs	Aucun message « ERROR » brut	Codes 400 / 404 / 413 / 405 / 502 + bandeaux explicites

3. Architecture de l'application

Le flux de traitement suit exactement l'architecture attendue : l'utilisateur saisit un indicateur, l'interface l'envoie à l'application Python, qui interroge une API externe, traite le JSON reçu, enregistre le résultat et l'affiche.

UTILISATEUR

```
|  saisit un IOC dans le formulaire
```

```
v
```

INTERFACE WEB (templates/index.html - Flask/Jinja2)

```
|  POST /analyze
```

```
v
```

APPLICATION PYTHON (app.py)

```
|-- ioc_parser.py ..... quel type d'IOC ? (IP / domaine / hash)
```

```
|-- enrichment.py ..... appel HTTP de l'API externe -> JSON
```

```
|-- scoring.py ..... signaux -> score 0-100 -> LOW/MEDIUM/HIGH/CRITICAL
```

```
|-- db.py ..... enregistrement dans Supabase (REST + JSON)
```

```
v
```

RESULTAT AFFICHE (rapport, JSON brut, historique, suppression)

Module	Rôle	Dépendance réseau
app.py	Routes Flask, enchaînement des étapes, gestion des erreurs HTTP	indirecte
ioc_parser.py	Détection et validation du type d'IOC, remise en forme (refang)	aucune
enrichment.py	Appels HTTP aux API externes et normalisation des réponses	API externes
scoring.py	Règles de risque : signaux vers score, niveau et verdict de réputation	aucune
db.py	Accès à Supabase : insertion, lecture, suppression, comptage	Supabase
templates/index.html	Interface : formulaire, rapport, historique	aucune
static/app.js	Confirmation avant suppression (séparé pour la	aucune

	politique CSP)	
--	----------------	--

Cette séparation en modules a un objectif concret : le calcul du score et la validation des entrées peuvent être testés sans Internet ni base de données, ce que font les 14 tests unitaires du projet.

4. Choix techniques et justification

Technologie	Rôle	Pourquoi ce choix
Python 3.11	Langage du projet	Langage imposé, riche en bibliothèques réseau
Flask 3.1	Interface web	Léger, routes HTTP explicites (GET/POST) et déploiement serverless possible
requests	Appels HTTP	Bibliothèque de référence, gestion native du timeout et des erreurs réseau
dotenv	Configuration	Aucune clé dans le code : les identifiants viennent de l'environnement
Supabase (PostgreSQL)	Base de données	Base demandée par le sujet, exposée en API REST : chaque requête est explicite
Vercel	Hébergement	Déploiement public gratuit en une commande, HTTPS automatique
pytest	Tests	Permet de vérifier la logique métier sans dépendance externe

Le choix de parler à Supabase via son API REST plutôt qu'avec une bibliothèque dédiée mérite une explication : la requête HTTP est entièrement visible dans le code — méthode, URL, en-têtes d'authentification, corps JSON — ce qui la rend démontrable et explicable, et allège considérablement le déploiement.

5. Les API externes

5.1 Sources retenues

Une source différente a été retenue pour chaque type d'indicateur, toutes gratuites et accessibles sans clé :

Type d'IOC	API	Données récupérées
Adresse IP	ip-api.com	Pays, ville, fournisseur, organisation, ASN, datacenter, proxy/VPN/Tor
Adresse IP (option)	AbuseIPDB	Score d'abus communautaire et nombre de signalements (clé API)
Nom de domaine	RDAP via rdap.org	Registrar, dates de création et d'expiration, statut, serveurs de noms
Empreinte	hashlookup.circl.lu	Nom du fichier, éditeur, indice de confiance (base CIRCL)

5.2 Anatomie d'un appel

Élément	Valeur (exemple : ip-api.com)
Méthode HTTP	GET
Endpoint	http://ip-api.com/json/185.220.101.1
Paramètres	fields=status,country,countryCode,city,isp,org,as,proxy,hosting,query
En-têtes envoyés	User-Agent: IOC-Enricher/1.0 (+url du dépôt) ; Accept: application/json
Clé API	aucune (offre gratuite, limitée à 45 requêtes par minute)
Délai maximum	8 secondes (timeout) pour ne pas bloquer l'utilisateur
Code de réponse attendu	200 OK

5.3 Exemple de réponse JSON et exploitation

```
{
  "status": "success", "country": "Germany", "countryCode": "DE",
  "city": "Berlin", "isp": "Stiftung Erneuerbare Freiheit",
  "org": "Tor Exit Node", "as": "AS205100 F3 Netze e.V.",
  "proxy": true, "hosting": true, "query": "185.220.101.1"
}
```

Ce JSON est converti en dictionnaire Python par `reponse.json()`, puis exploité :

```
reponse = requests.get(url, headers=HEADERS, params=params, timeout=8) # appel HTTP

if reponse.status_code >= 400: # code de statut
    raise EnrichmentError(f"HTTP {reponse.status_code}", kind="http")

brut = reponse.json() # JSON -> dictionnaire

if brut.get("proxy"): # signal de risque
    signaux.append({"points": 35, "label": "Adresse detectee comme proxy / VPN / Tor"})
```

5.4 Codes de statut HTTP gérés

Code	Situation	Comportement de l'application
200	Réponse normale	Les données sont affichées puis enregistrées
401	Clé API refusée (AbuseIPDB)	Message explicite et repli automatique sur ip-api.com
404	Domaine ou empreinte inconnus du registre	Traité comme un signal de risque, pas comme une erreur
429	Quota dépassé	Message « quota atteint, réessayez dans une minute »
5xx	Panne du fournisseur	Message d'API injoignable, l'application reste utilisable

6. Base de données

6.1 Pourquoi une base de données ?

Une base de données répond à trois besoins : conserver la trace des analyses après la fermeture du navigateur, savoir si un indicateur a déjà été étudié, et permettre la suppression d'une entrée devenue obsolète. Le stockage est assuré par PostgreSQL, hébergé et exposé par Supabase.

6.2 Table ioc_analyses

Colonne	Type	Description
id	bigint (identité)	Clé primaire, générée automatiquement
ioc	text	L'indicateur analysé
ioc_type	text	ip, domain ou hash
risk_level	text	LOW, MEDIUM, HIGH ou CRITICAL
risk_score	integer	Score de 0 à 100
source_api	text	API qui a fourni l'enrichissement
summary	jsonb	Champs enrichis et endpoint appelé
reasons	jsonb	Liste des raisons du score
created_at	timestamptz	Date de l'analyse (now() par défaut)

6.3 Comment les données sont envoyées

Supabase expose chaque table à travers une API REST (PostgREST). L'envoi d'une analyse est donc une requête HTTP ordinaire :

```
POST https://<projet>.supabase.co/rest/v1/ioc_analyses
```

```
apikey: <cle>          Authorization: Bearer <cle>
```

```
Content-Type: application/json    Prefer: return=representation
```

```
{"ioc": "185.220.101.1", "ioc_type": "ip", "risk_level": "HIGH", "risk_score": 70, ...}
```

L'en-tête `Prefer: return=representation` demande à la base de renvoyer la ligne créée : l'application récupère ainsi son identifiant et sa date, affichés dans le rapport.

6.4 Sécurité de l'accès

Les règles RLS (Row Level Security, ou sécurité au niveau des lignes) sont activées sur la table et n'autorisent aucun accès public. L'application se connecte avec une clé secrète conservée côté serveur : elle ne transite jamais vers le navigateur de l'utilisateur. L'URL du projet et sa clé publique ne permettent donc ni de lire, ni d'écrire, ni de supprimer une analyse.

7. Interface web

L'interface a été conçue comme un poste d'analyse : fond sombre, information essentielle mise en avant, aucune ambiguïté sur le niveau de risque.

IOC ENRICHER — Analyst Desk

Enrichissement d'indicateurs de compromission — adresse IP, nom de domaine, empreinte de fichier · Base de données connectée

ANALYSER UN INDICATEUR

185.10.10.5 · evil-domain.com · 44d88612fea8a8f36de82e1278abb02f

Enrichir

Formats acceptés :

adresse IP (v4/v6), nom de domaine, empreinte MD5 / SHA1 / SHA256. Les IOC neutralisés sont acceptés (185[.]10[.]10[.]5 ou hxxps://evil[.]com).

HISTORIQUE DES ANALYSES

#	IOC	TYPE	RÉPUTATION	RISQUE	SCORE	SOURCE API	DATE (UTC)	
37	44d88612fea8a8f36de82e1278abb02f	Empreinte de fichier	SAIN	LOW	0	CIRCL hashlookup	2026-09-16 10:55:49	Supprimer
36	github.com	Nom de domaine	SAIN	LOW	0	RDAP (rdap.org)	2026-09-16 10:55:48	Supprimer
35	185.220.101.1	Adresse IP	SUSPECT	HIGH	70	ip-api.com	2026-09-16 10:55:47	Supprimer

3 analyse(s) affichée(s) — les 25 plus récentes.

IOC Enricher — Flask · API externes : ip-api.com, RDAP (rdap.org), CIRCL hashlookup

Figure 1 — Formulaire de saisie et historique des analyses.

Page 9 of 18

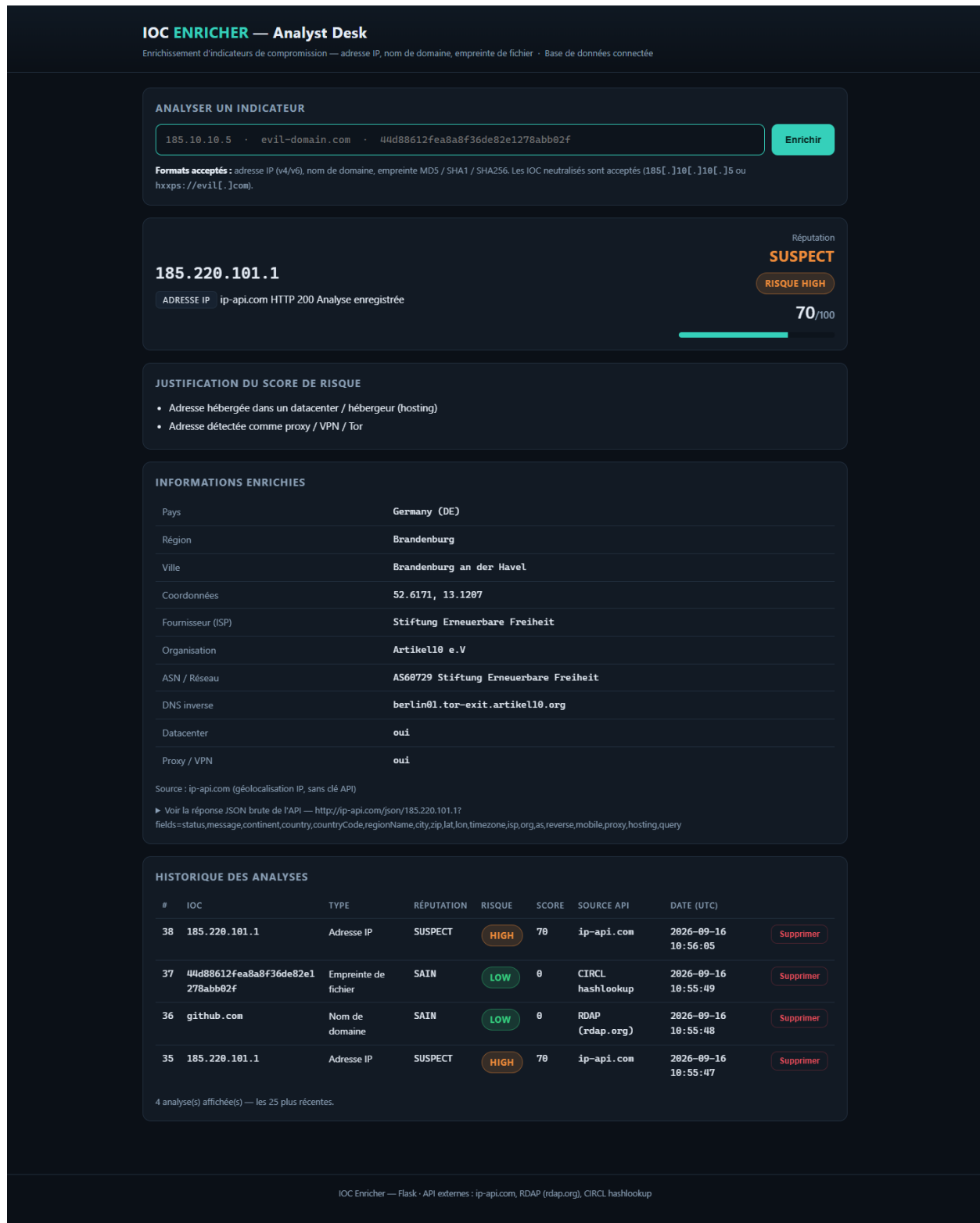


Figure 2 — Rapport d'analyse d'une adresse IP : verdict de réputation, niveau et score de risque justifié, informations enrichies.

IOC ENRICHER — Analyst Desk

Enrichissement d'indicateurs de compromission — adresse IP, nom de domaine, empreinte de fichier · Base de données connectée

ANALYSER UN INDICATEUR

185.10.10.5 · evil-domain.com · 44d88612fea8a8f36de82e1278abb02f

Enrichir

Formats acceptés :

adresse IP (v4/v6), nom de domaine, empreinte MD5 / SHA1 / SHA256. Les IOC neutralisés sont acceptés (185[.]10[.]10[.]5 ou hxxps://evil[.]com).

185.220.101.1

ADRESSE IP · ip-api.com HTTP 200 Analyse enregistrée

Réputation

SUSPECT

RISQUE HIGH

70/100

JUSTIFICATION DU SCORE DE RISQUE

- Adresse hébergée dans un datacenter / hébergeur (hosting)
- Adresse détectée comme proxy / VPN / Tor

INFORMATIONS ENRICHIES

Pays	Germany (DE)
Région	Brandenburg
Ville	Brandenburg an der Havel
Coordonnées	52.6171, 13.1207
Fournisseur (ISP)	Stiftung Erneuerbare Freiheit
Organisation	Artikel10 e.V
ASN / Réseau	AS60729 Stiftung Erneuerbare Freiheit
DNS inverse	berlin01.tor-exit.artikel10.org
Datacenter	oui
Proxy / VPN	oui

Source : ip-api.com (géolocalisation IP, sans clé API)

▼ Voir la réponse JSON brute de l'API — <http://ip-api.com/json/185.220.101.1?fields=status,message,continent,country,countryCode,regionName,city,zip,lat,lon,timzone,isp,org,as,reverse,mobile,proxy,hosting,query>

```
{
  "as": "AS60729 Stiftung Erneuerbare Freiheit",
  "city": "Brandenburg an der Havel",
  "continent": "Europe",
  "country": "Germany",
  "countryCode": "DE",
  "hosting": true,
  "isp": "Stiftung Erneuerbare Freiheit",
  "lat": 52.6171,
  "lon": 13.1207,
  "mobile": false,
  "org": "Artikel10 e.V",
  "proxy": true,
  "query": "185.220.101.1",
  "regionName": "Brandenburg",
  "reverse": "berlin01.tor-exit.artikel10.org",
  "status": "success",
  "timzone": "Europe/Berlin",
  "zip": "14621"
}
```

HISTORIQUE DES ANALYSES

#	IOC	TYPE	RÉPUTATION	RISQUE	SCORE	SOURCE API	DATE (UTC)	
38	185.220.101.1	Adresse IP	SUSPECT	HIGH	70	ip-api.com	2026-09-16 10:56:05	Supprimer
37	44d88612fea8a8f36de82e1278abb02f	Empreinte de fichier	SAIN	LOW	0	CIRCL hashlookup	2026-09-16 10:55:49	Supprimer
36	github.com	Nom de domaine	SAIN	LOW	0	RDAP (rdap.org)	2026-09-16 10:55:48	Supprimer
35	185.220.101.1	Adresse IP	SUSPECT	HIGH	70	ip-api.com	2026-09-16 10:55:47	Supprimer

4 analyse(s) affichée(s) — les 25 plus récentes.

IOC Enricher — Flask · API externes : ip-api.com, RDAP (rdap.org), CIRCL hashlookup

Page 11 of 18

Figure 3 — Réponse JSON brute de l'API, affichée pour la démonstration.

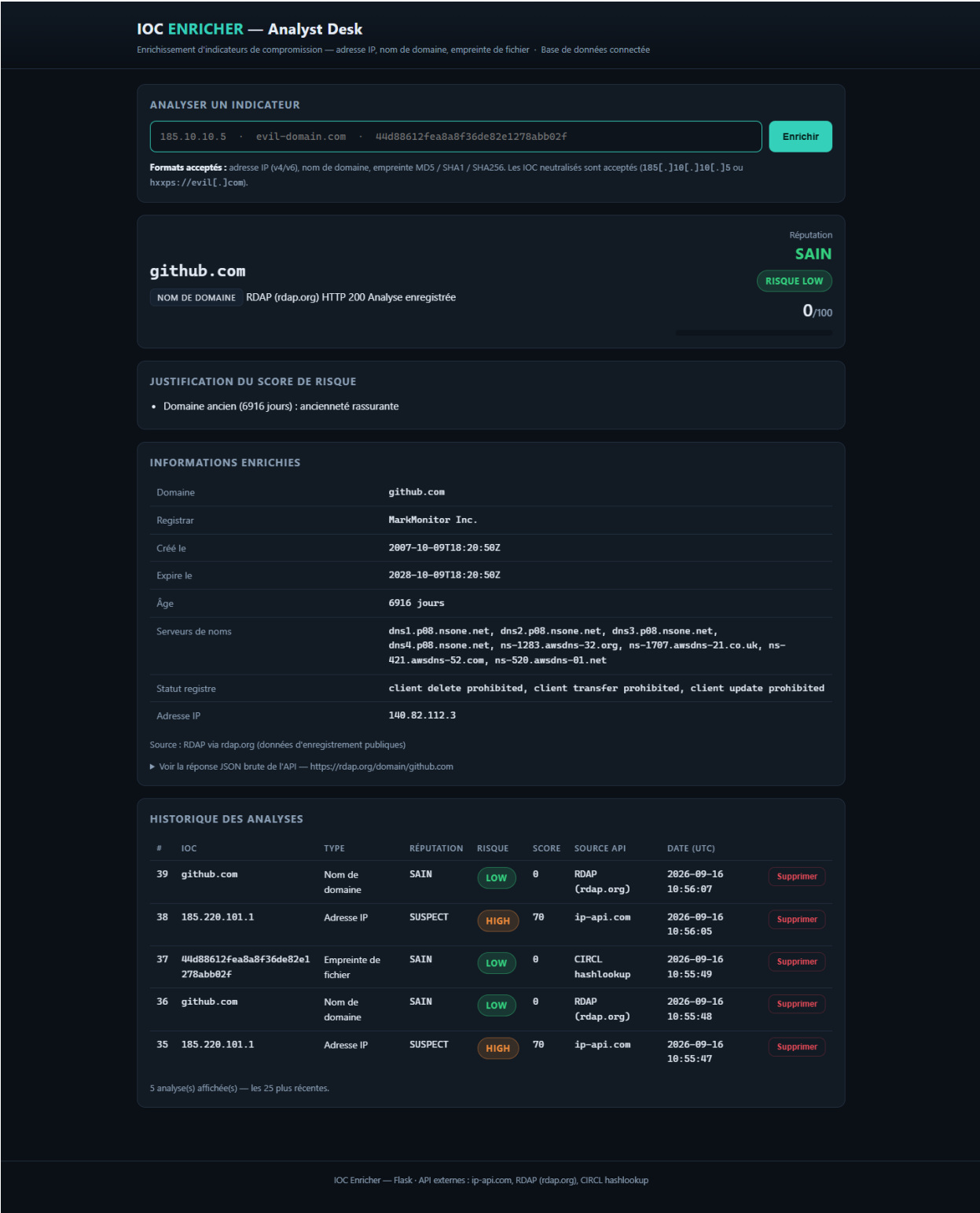


Figure 4 — Analyse d'un nom de domaine (enregistrement RDAP et résolution DNS).

8. Déploiement

8.1 Publication du code

Le code est versionné avec Git et publié sur GitHub. Le fichier .gitignore exclut le fichier .env : aucune clé n'est présente dans le dépôt, y compris dans l'historique des commits. Un fichier .env.example documente les variables attendues sans en révéler la valeur.

8.2 Hébergement

L'application est déployée sur Vercel. Vercel exécute le fichier api/index.py, qui importe l'application Flask définie à la racine du projet. Le fichier vercel.json précise les fichiers à embarquer dans la fonction ainsi qu'un délai d'exécution maximal de 30 secondes.

```
vercel link --project ioc-enricher
```

```
vercel env add SUPABASE_URL production
```

```
vercel env add SUPABASE_SECRET_KEY production
```

```
vercel --prod
```

8.3 Variables d'environnement

Variable	Environnement	Rôle
SUPABASE_URL	local (.env) et Vercel	Adresse de l'API REST du projet
SUPABASE_SECRET_KEY	local (.env) et Vercel	Clé secrète utilisée côté serveur uniquement
ABUSEIPDB_API_KEY	optionnelle	Active la réputation d'abus des adresses IP

En local, ces variables sont lues depuis le fichier .env ; en production, elles sont saisies dans le tableau de bord Vercel. Dans les deux cas, elles sont absentes du code source.

9. Sécurité : audit et corrections

Une revue de sécurité a été menée sur les trois briques du projet — dépôt, base de données et hébergement — puis les faiblesses identifiées ont été corrigées.

Faiblesse identifiée	Correction apportée
Table de base de données ouverte en lecture, insertion et suppression au rôle public : la clé publique suffisait à vider la base	Accès serveur par clé secrète ; politiques ouvertes supprimées (RLS fermées)
Bibliothèque requests 2.32.3 vulnérable (CVE-2024-47081 : fuite des identifiants netrc)	Mise à jour vers requests 2.34.2
Versions anciennes de Flask et python-dotenv	Passage à Flask 3.1.3 (Werkzeug 3.1.8) et python-dotenv 1.2.3
Aucun en-tête de sécurité HTTP	CSP restrictive, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, X-Robots-Tag
Déploiements de préversion accessibles publiquement	Protection d'accès activée sur les préversions, production publique
Route /health exposant la configuration et le nombre d'analyses	Réponse réduite au strict minimum
Saisie non bornée	Limite de 512 caractères et corps de requête limité à 16 Ko (erreur 413 gérée)
Code JavaScript en ligne, incompatible avec une	Script déplacé dans static/app.js

politique de sécurité stricte	
Alertes de sécurité GitHub désactivées, branche non protégée	Dependabot (alertes et mises à jour), analyse des secrets, push protection, protection de la branche master

Un audit automatisé des dépendances (pip-audit) ne remonte plus aucune vulnérabilité connue après ces mises à jour.

La fermeture de la base a ensuite été vérifiée depuis l'extérieur, en rejouant l'attaque avec la clé publique du projet :

Tentative avec la clé publique	Résultat obtenu
Lire toute la table	HTTP 200 — 0 ligne visible
Insérer une ligne	HTTP 401 — new row violates row-level security policy
Supprimer toute la table	HTTP 204, mais 0 ligne affectée
Les analyses déjà enregistrées	Toujours présentes après les trois tentatives

Pendant ces tentatives, l'application continuait de fonctionner normalement grâce à la clé secrète conservée côté serveur : analyse d'un indicateur, enregistrement, affichage de l'historique et suppression d'une entrée.

10. Tests et validation

10.1 Tests automatisés

- 16 tests unitaires (pytest) : détection du type d'IOC, remise en forme, saisie vide, saisie invalide, normalisation, fourchettes de score, verdict de réputation et bornage 0-100. Aucun accès réseau ni base de données requis.
- 6 scénarios d'appel réel aux API externes : 6 réussites sur 6.
- 31 vérifications de bout en bout sur l'application publique : pages, en-têtes de sécurité, analyse d'une IP, d'un domaine et d'une empreinte, verdict de réputation, écriture puis relecture en base, suppression, cas d'erreur.

10.2 Résultats obtenus sur des indicateurs réels

Indicateur	API	Code HTTP	Réputation	Niveau et score
8.8.8.8	ip-api.com	200	DOUTEUX	MEDIUM (35/100)
185.220.101.1	ip-api.com	200	SUSPECT	HIGH (70/100)
github.com	RDAP (rdap.org)	200	SAIN	LOW (0/100)
ce-domaine-nexiste-pas-1234567890.com	RDAP (rdap.org)	404	SUSPECT	HIGH (55/100)
44d88612fea8a8f36de82e1278abb02f	CIRCL hashlookup	200	SAIN	LOW (0/100)
00000000000000000000000000000000	CIRCL hashlookup	404	DOUTEUX	MEDIUM (30/100)

Le score obtenu est traduit en deux informations complémentaires, affichées côte à côte dans le rapport d'analyse : une réputation, qui est la conclusion lisible immédiatement, et un niveau de risque, qui en est la gradation technique.

Score	Niveau de risque	Réputation affichée
-------	------------------	---------------------

75 à 100	CRITICAL	MALVEILLANT
50 à 74	HIGH	SUSPECT
25 à 49	MEDIUM	DOUTEUX
0 à 24	LOW	SAIN

10.3 Les six cas exigés

Cas	Test	Résultat observé
Cas normal	Saisir 8.8.8.8	Rapport complet, analyse enregistrée
Cas invalide	Saisir « bonjour les amis »	HTTP 400 et message « n'est pas un IOC valide »
Cas vide	Valider sans rien saisir	HTTP 400 et message « Aucune valeur saisie »
Erreur API	Saisir une IP privée 192.168.1.10	HTTP 502 et message « private range », aucune trace Python
Erreur réseau	Couper Internet puis analyser	Message d'API injoignable, application toujours utilisable
Erreur base de données	Retirer la configuration de la base	Le rapport s'affiche, bandeau « analyse non enregistrée »

11. Difficultés rencontrées et solutions

Difficulté	Cause	Solution retenue
Les gabarits HTML n'étaient pas trouvés après déploiement	Les fichiers hors du dossier de la fonction ne sont pas embarqués par défaut	Déclaration explicite des fichiers à inclure dans vercel.json
L'URL de production renvoyait une page de connexion	Protection d'accès active sur tous les déploiements	Protection limitée aux préversions, production laissée publique
Les en-têtes de sécurité n'étaient pas appliqués	La section headers est ignorée lorsque la configuration utilise routes	En-têtes ajoutés directement par l'application, puis vérifiés en ligne
ip-api.com renvoie toujours le code 200, même en cas d'échec	Le statut réel se trouve dans le corps JSON	Lecture du champ status et conversion en erreur explicite
Une IP privée ne produit aucun enrichissement	Les bases publiques ne connaissent pas les plages privées	Détection en amont et message pédagogique à l'utilisateur

12. Améliorations envisagées

- Ajouter une source de réputation pour les empreintes de fichiers (VirusTotal) en complément de CIRCL.
- Mettre en place une authentification légère pour réserver l'historique à l'analyste.
- Ajouter une limitation du nombre de requêtes par adresse pour protéger les quotas des API.
- Exporter l'historique au format CSV pour l'intégrer à un outil de suivi.
- Afficher une carte pour les indicateurs géolocalisés, comme dans un outil de renseignement.
- Envoyer une alerte automatique lorsqu'un indicateur dépasse un niveau de risque donné.

13. Conclusion

Le projet atteint l'objectif fixé : une application Python fonctionnelle, dotée d'une interface web, qui interroge réellement trois API externes, exploite des données JSON, enregistre ses analyses dans une

base de données PostgreSQL hébergée sur Supabase, et reste accessible publiquement à l'adresse <https://ioc-enricher-delta.vercel.app>.

Au-delà du fonctionnement attendu, le projet a été mené jusqu'à des préoccupations de production : gestion explicite de tous les cas d'erreur, absence totale de secrets dans le code et dans l'historique du dépôt, durcissement de la base de données, en-têtes de sécurité HTTP, et vérification automatisée de chaque étape sur l'application en ligne. C'est cette dernière exigence — livrer un produit qui fonctionne réellement et que l'on sait expliquer — qui a guidé l'ensemble des choix techniques.

Annexe A — Arborescence du projet

ioc-enricher/

app.py	application Flask, routes et gestion des erreurs
ioc_parser.py	detection et validation du type d'IOC
enrichment.py	appels aux API externes et normalisation du JSON
scoring.py	regles de risque et calcul du score
db.py	acces Supabase (API REST PostgREST)
templates/index.html	interface web
static/app.js	confirmation de suppression
api/index.py	point d'entree du deploiement Vercel
sql/schema.sql	creation de la table et regles de securite
tests/test_socle.py	14 tests unitaires
tools/smoke_api.py	test des API en conditions reelles
requirements.txt	.env.example
.gitignore	vercel.json
README.md	SECURITY.md

Annexe B — Extraits de code commentés

Détection du type d'indicateur (ioc_parser.py) :

```
try:                                # est-ce une adresse IP ?
    ipaddress.ip_address(candidate)
    return "ip"
except ValueError:
    pass

if len(candidate) in HASH_ALGORITHMS and HEX_RE.match(candidate):
    return "hash"                    # 32, 40 ou 64 caracteres hexadecimaux
```

Calcul du score (scoring.py) :

```
total = BASE_SCORE + sum(int(s.get("points", 0)) for s in signaux)
total = max(0, min(100, total))      # score borne entre 0 et 100
niveau = level_for(total)            # LOW / MEDIUM / HIGH / CRITICAL
```

Écriture dans la base (db.py) :

```
reponse = requests.request(method, _endpoint(), params=params, json=payload,
                             headers=_headers(extra_headers), timeout=TIMEOUT)
```

Annexe C — Commandes utiles

```
pip install -r requirements.txt      # installer les dependances
python app.py                       # lancer en local (http://127.0.0.1:5000)
python -m pytest -v                 # lancer les 14 tests unitaires
python tools/smoke_api.py           # tester les API externes en reel
vercel --prod                       # deployer en production
```